DD: FUL-02-STEP-SUBSCRIPTION — Subscription Creation

Developer implementation guide: `DD_FUL-02-STEP-SUBSCRIPTION-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_FUL-02-STEP-SUBSCRIPTION-v1.0.md`.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md The step that creates the Subscription record in SUB-LM-01. In the KE WIK flow, runs right after validation (so payment can be tied to a known subscription). The worker is one short call to SUB-LM-01. No dedicated phase (transitive — phase before this step persists across this step).

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-02-STEP-SUBSCRIPTION-v1.0.md`
Version	`v1.0`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `GET /api/subscriptions`
- `GET /api/subscriptions?customerId=...&homepassId=...&packageRef=...`
- `POST /api/subscriptions`
- `POST /engine-rest/external-task/ext_t_01HZX9K8M7Q2N4P6R3T5W8Y0Z4/complete`
- `POST /engine-rest/external-task/fetchAndLock`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `compute-expected-payment`
- `create-subscription`
- `order-create-subscription`
- `validate-order`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `PaymentAppliedConsumer`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.

Concern	Recommendation
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md

The step that creates the Subscription record in SUB-LM-01. In the KE WIK flow, runs right after validation (so payment can be tied to a known subscription). The worker is one short call to SUB-LM-01.

No dedicated phase (transitive — phase before this step persists across this step).

1. What this step does

The Subscription is the long-lived domain object the customer pays for — the persistent record of "this customer has this Package at this HomePass." Until this step runs, the order exists but there is no Subscription yet; downstream systems (billing engine, dunning engine, network provisioning) have nothing to operate on. This step turns the validated order into a real Subscription row in SUB-LM-01's catalog.

The work is just one HTTP call: `POST /api/subscriptions` to SUB-LM-01 with the customer, package, HomePass, and operator-default billing parameters. SUB-LM-01 inserts a row in its `subscription` table with `status_code = CREATED`, returns the new `subscriptionId`. The worker stores that ID onto `customer_order.subscription_id` and completes the Camunda external task. That's it.

The Subscription is in `CREATED` status — not yet `ACTIVE`. Activation happens later, in STEP-ACTIVATION, when SUB-WF-ACTIVATE-01 is called and it walks the subscription through `CREATED → PENDING_ACTIVATION → ACTIVE` after billing charges and network provisioning succeed. This step deliberately doesn't activate; it just registers.

Position in the KE WIK flow

WIK BPMN sequences: validate → create subscription → compute payment → notify payment instructions → wait payment → KYC L1 → KYC Team Leader → create install WO → wait finalize → trigger activation. Subscription exists before payment is requested. The customer is paying *for a specific subscription*; the payment reference in BIL-01 can be tied to `subscription_id` cleanly when activation later charges installation + first cycle.

What this step deliberately doesn't do

- **Doesn't activate**. CREATED status, no network provisioning, no billing charges. STEP-ACTIVATION owns the activation handoff.
- **Doesn't emit a Kafka event**. SUB-LM-01 emits `sophix.subscription.subscription.created` on its `POST /api/subscriptions`; FUL-02 does not publish a duplicate.
- **Doesn't set** `cycle_anchor_day` (for CALENDAR_ANCHOR cycle model). SUB-LM-01 accepts the field as optional on creation; the value is resolved by SUB-WF-ACTIVATE-01 at activation time per R-SUB-LM-01-S-6a (operator overflow policy applies when the customer's intended activation date is day 29/30/31).
- **Doesn't change** `current_phase`. The phase before this step (`IN_VALIDATION`) persists across this step. STEP-PAYMENT transitions to `AWAITING_PAYMENT`.

2. Worked example end-to-end (KE WIK)

Continuing the same scenario from STEP-VALIDATE: order `ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`, customer `cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6`, package `pkg_INET_VOIP_RES`, homepass `hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P`, validated at 09:00:01. Camunda has just advanced the BPMN past

`validate-order` to the next external task: `create-subscription`.

Step 1 — Camunda offers the external task on topic `order-create-subscription`

The worker pool's `CreateSubscriptionWorker` long-poll picks up the task:

```
POST /engine-rest/external-task/fetchAndLock HTTP/1.1
Content-Type: application/json

{
  "workerId": "fulfillment-order-capture-worker-5",
  "asyncResponseTimeout": 30000,
  "maxTasks": 10,
  "topics": [
    { "topicName": "order-create-subscription", "lockDuration": 15000 }
  ]
}
```

Camunda hands over:

```
[
  {
    "id":                "ext_t_01HZX9K8M7Q2N4P6R3T5W8Y0Z4",
    "topicName":         "order-create-subscription",
    "processInstanceId": "cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "businessKey":       "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "variables": {
      "orderId":      { "value": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1", "type": "String" },
      "customerId":   { "value": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6", "type": "String" },
      "packageRef":   { "value": "pkg_INET_VOIP_RES",               "type": "String" },
      "homepassId":   { "value": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",   "type": "String" },
      "operatorCode": { "value": "WIK",                             "type": "String" }
    },
    "lockExpirationTime": "2026-05-18T09:00:16.234Z"
  }
]
```

The worker has 15 seconds; SUB-LM-01 typically responds in <200 ms.

Step 2 — Worker reads operator defaults from `ful_02_config`

```
SELECT default_billing_mode, default_cycle_model
  FROM ful_02_config
 WHERE operator_code = 'WIK';
```

→ `default_billing_mode = 'POSTPAID'`, `default_cycle_model = 'CALENDAR_ANCHOR'`.

Step 3 — Worker POSTs to SUB-LM-01

```
POST /api/subscriptions HTTP/1.1
Host: sub-lm-01.sophix.internal
Authorization: Bearer <module-svc-creds>
Content-Type: application/json

{
  "customerId":   "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "packageRef":   "pkg_INET_VOIP_RES",
  "homepassId":   "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "billingMode":  "POSTPAID",
  "cycleModel":   "CALENDAR_ANCHOR",
  "cycleFrequencyMonths": 1,
  "startDate":    "2026-05-18"
}
```

Notes on the body:

- No `operatorCode` field. SUB-LM-01 derives operator scope from `customerId` (each customer belongs to one operator tenant). FUL-02 doesn't need to pass it again.
- No `cycleAnchorDay`. CALENDAR_ANCHOR requires it at activation time, but per R-SUB-LM-01-S-6a it's resolved by SUB-WF-ACTIVATE-01 (STEP-ACTIVATION), not here. SUB-LM-01 leaves it NULL on the row during CREATED.
- `startDate` is the intended service-start date; here defaulted to "today" since the request had `intendedActivationDate = null` at STEP-CAPTURE.

SUB-LM-01 inserts a row in `subscription` with `status_code = CREATED` and returns:

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
{
  "subscriptionId":  "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
  "customerId":      "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "packageRef":      "pkg_INET_VOIP_RES",
  "packageVersion":  "pkv_01HZ_INET_VOIP_RES_v3",
  "homepassId":      "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "billingMode":     "POSTPAID",
  "cycleModel":      "CALENDAR_ANCHOR",
  "cycleAnchorDay":  null,
  "cycleFrequencyMonths": 1,
  "startDate":       "2026-05-18",
  "statusCode":      "CREATED",
  "currency":        "KES",
  "createdAt":       "2026-05-18T09:00:02.103Z"
}
```

SUB-LM-01 also internally publishes `sophix.subscription.subscription.created` on Kafka — its own event, independently consumed by CVM, search index, analytics. The worker doesn't need to wait for or care about that emission.

Step 4 — Worker updates `customer_order` with the new subscription_id

```
UPDATE customer_order
   SET subscription_id = 'sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q',
       last_updated_at = NOW()
 WHERE order_id        = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

Step 5 — Worker reports completion to Camunda

```
POST /engine-rest/external-task/ext_t_01HZX9K8M7Q2N4P6R3T5W8Y0Z4/complete HTTP/1.1
Content-Type: application/json

{
  "workerId": "fulfillment-order-capture-worker-5",
  "variables": {
    "subscriptionId": { "value": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q", "type": "String" }
  }
}
```

Camunda 204. BPMN advances to the next external task — in WIK, that's `compute-expected-payment` (STEP-PAYMENT).

Total elapsed inside this step: ~250 ms.

3. Worked example — re-fetch after lock expiry (idempotent recovery)

Rare but possible: the worker calls SUB-LM-01 at step 3, SUB-LM-01 successfully inserts the row, but the response is lost (network hiccup, worker crash before it processed the response). The worker's lock with Camunda expires; Camunda releases the task; another worker instance picks it up.

The second worker repeats step 3:

```
POST /api/subscriptions
{ ... same body ... }
```

SUB-LM-01's `subscription` table has a UNIQUE constraint on `(customer_id, homepass_id, package_ref)` for non-terminated subscriptions (per R-SUB-LM-01-S-1). The INSERT fails the constraint. SUB-LM-01 responds:

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "error":          "DUPLICATE_ACTIVE_SUBSCRIPTION",
  "detail":         "An active subscription already exists for this (customer, homepass, package) tuple",
  "existingSubscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q"
}
```

The worker recognises the 409 response, extracts `existingSubscriptionId`, treats it as success: updates `customer_order.subscription_id`, completes the Camunda task. No duplicate subscription was created.

If SUB-LM-01's 409 response doesn't carry `existingSubscriptionId` (older API version), the worker does a follow-up `GET /api/subscriptions?customerId=...&homepassId=...&packageRef=...` to resolve the ID. Same end-state.

4. Worker

4.1 order-create-subscription

Field	Value
Topic	`order-create-subscription`
Purpose	Create a Subscription record in SUB-LM-01 in CREATED status; capture the new `subscriptionId` onto `customer_order`.
Process variables read	`orderId`, `customerId`, `packageRef`, `homepassId`, `operatorCode`
Config read	`ful_02_config.default_billing_mode` (`POSTPAID` for WIK), `ful_02_config.default_cycle_model` (`CALENDAR_ANCHOR` for WIK)
External HTTP calls	SUB-LM-01 `POST /api/subscriptions`. On 409, optional follow-up `GET /api/subscriptions` to resolve existing ID.
Process variables written	`subscriptionId`
`customer_order` columns written	`subscription_id`
Kafka emissions	None — SUB-LM-01 emits `sophix.subscription.subscription.created` on its own.
Lock duration	15 000 ms
Retry policy	0 retries on `/failure`. The worker handles 409 (duplicate) internally as success; everything else either succeeds first try or is genuinely broken.
Idempotency	Safe to re-fetch. SUB-LM-01's UNIQUE constraint on `(customer_id, homepass_id, package_ref)` prevents duplicate rows; the worker treats 409 as success after resolving `existingSubscriptionId`.

Failure modes:

- **SUB-LM-01 domain error** (`PACKAGE_NOT_AVAILABLE`, `CUSTOMER_INELIGIBLE`, `HOMEPASS_NOT_SELLABLE`) → worker reports `/failure` with the error. This shouldn't normally happen because STEP-VALIDATE already checked these — if it does, it usually means STEP-VALIDATE's checks and SUB-LM-01's checks diverged (catalog change between the two steps). Raises a Camunda incident; ops investigates.
- **Network failure / SUB-LM-01 unreachable** → `/failure` with `UPSTREAM_UNREACHABLE`. Raises incident.
- **409 duplicate** → handled internally as success, see §3 above. Not a worker failure.

5. DB tables touched

customer_order (FRAMEWORK §5.1) — UPDATE on the existing row

Column	When set	Set by	Example value
`subscription_id`	After SUB-LM-01 returns 201	`order-create-subscription` worker	`sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q`
`current_activity_id`	On Camunda transition	`PhaseProjector`	`create-subscription` → `compute-expected-payment`
`last_updated_at`	On UPDATE	trigger / worker	`2026-05-18 09:00:02.103`

`current_phase` is NOT changed by this step. The phase before it (`IN_VALIDATION`) persists until STEP-PAYMENT moves it to `AWAITING_PAYMENT`.

Sample data — three orders post-subscription-creation:

order_id	operator_code	sales_channel	subscription_id	current_phase	current_activity_id	last_updated_at
`ord_01HZ...A`	`WIK`	`FIELD_AGENT_PWA`	`sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q`	`IN_VALIDATION` (in transit to AWAITING_PAYMENT)	`compute-expected-payment`	2026-05-18 09:00:02.103
`ord_01HZ...B`	`WIK`	`BACKOFFICE`	`sub_01HX_LAVINGTON_42`	`IN_VALIDATION` (in transit to AWAITING_PAYMENT)	`compute-expected-payment`	2026-05-18 09:30:17.502
`ord_01HZ...H`	`WIK`	`CALL_CENTER`	`sub_01HX_WESTLANDS_05`	`IN_VALIDATION` (in transit to AWAITING_PAYMENT)	`compute-expected-payment`	2026-05-18 09:45:33.871

subscription (SUB-LM-01's table) — INSERT (owned by SUB-LM-01)

This step doesn't write to SUB-LM-01's table directly; SUB-LM-01 owns it. Shown here for traceability of what gets created on SUB-LM-01's side as a result of this step. Full schema in DD_SUB-LM-01.

Column	Example value after STEP-SUBSCRIPTION
`id`	`sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q`
`customer_id`	`cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6`
`package_ref`	`pkg_INET_VOIP_RES`
`package_version`	`pkv_01HZ_INET_VOIP_RES_v3` (pinned to ACTIVE version at creation per R-SUB-LM-01-S-2)
`homepass_id`	`hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P`
`billing_mode`	`POSTPAID`
`cycle_model`	`CALENDAR_ANCHOR`
`cycle_anchor_day`	NULL (resolved at activation by SUB-WF-ACTIVATE-01 per R-SUB-LM-01-S-6a)
`cycle_frequency_months`	1
`start_date`	2026-05-18
`status_code`	`CREATED`
`currency`	`KES`
`activated_at`	NULL (set at first transition to ACTIVE)
`created_at`	2026-05-18 09:00:02.103

customer_order_phase_history (FRAMEWORK §5.2) — appended

One row appended by `PhaseProjector` on each Camunda activity transition:

Column	Example value (WIK)
`id`	`ordh_01HZX9K8M7Q2N4P6R3T5W8Y0Z4`
`order_id`	`ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`
`previous_phase`	`IN_VALIDATION`
`new_phase`	`IN_VALIDATION` (no phase change — subscription creation is transitive)
`previous_activity_id`	`create-subscription`
`new_activity_id`	`compute-expected-payment`
`transition_at`	`2026-05-18 09:00:02.345`
`triggered_by`	`worker_completion`
`triggered_by_detail`	`CreateSubscriptionWorker`
`metadata`	`{"subscriptionId":"sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q","billingMode":"POSTPAID","cycleModel":"CALENDAR_ANCHOR"}`

6. Kafka events emitted

None by this step. The Subscription's creation is announced by SUB-LM-01 itself on the topic `sophix.subscription.subscription.created` (see DD_SUB-LM-01 §events). Downstream consumers that care about "this customer has a new Subscription" (search index, CVM, analytics) subscribe to SUB-LM-01's topic, not to anything FUL-02 publishes.

The split is deliberate: events naming Subscription state are SUB-LM-01's responsibility (single source of truth — see R-FUL-02-SUB-5). FUL-02 only emits Order-named events.

7. Step-pinned rules

ID	Rule
R-FUL-02-SUB-1	`order-create-subscription` calls SUB-LM-01 `POST /api/subscriptions` with operator defaults from `ful_02_config` (`default_billing_mode`, `default_cycle_model`). No `operatorCode` or `originatingOrder` is passed in the body — SUB-LM-01 derives operator scope from `customerId`; the `customer_order ↔ subscription` linkage lives on the FUL-02 side via `customer_order.subscription_id`.
R-FUL-02-SUB-2	Worker is operator-agnostic. KE WIK BPMN invokes it early (post-validate, pre-payment). The worker doesn't depend on the BPMN's choice of position; future operators with different sequences would reuse the same worker.
R-FUL-02-SUB-3	Idempotent re-fetch: SUB-LM-01's 409 on `(customer_id, homepass_id, package_ref)` UNIQUE constraint is treated as success — the worker extracts `existingSubscriptionId` (or fetches it via follow-up GET if missing), updates `customer_order.subscription_id`, completes the Camunda task. No duplicate subscriptions are ever created.
R-FUL-02-SUB-4	After this step completes, `customer_order.subscription_id` is non-NULL for the rest of the order's life. Downstream consumers (STEP-PAYMENT's `PaymentAppliedConsumer`, STEP-INSTALL, STEP-ACTIVATION) can safely assume the subscription exists.
R-FUL-02-SUB-5	SUB-LM-01's `sophix.subscription.subscription.created` is the single source of truth for "Subscription exists." STEP-SUBSCRIPTION does not publish a redundant FUL-02 event.
R-FUL-02-SUB-6	Subscription is created in `CREATED` status — not `ACTIVE`. Activation is handed off to SUB-WF-ACTIVATE-01 by STEP-ACTIVATION later in the flow. This step never activates.
R-FUL-02-SUB-7	`cycle_anchor_day` is NOT passed by this step (even for CALENDAR_ANCHOR mode). Per R-SUB-LM-01-S-6a, SUB-WF-ACTIVATE-01 resolves the anchor day at activation time with operator-specific overflow policy (for customers whose intended activation date is day 29/30/31). SUB-LM-01 accepts the column as NULL on the CREATED row.
R-FUL-02-SUB-8	Domain errors from SUB-LM-01 (`PACKAGE_NOT_AVAILABLE`, `CUSTOMER_INELIGIBLE`, etc.) shouldn't normally fire because STEP-VALIDATE already covered these checks. If they do, it's a sign STEP-VALIDATE's snapshot diverged from SUB-LM-01's current state (catalog change between the two steps); worker raises a Camunda incident for ops review rather than auto-rejecting the order.